

目次

1. チュートリアル説明	2
2. Pybit 紹介	2
2.2 仕様	3
2.3 部品説明	4
3. Pybit の取り付け	5
4. Python エディタ 必須知識	6
4.1 新規プロジェクトの作成	7
4.2 プロジェクトのローカル保存	8
4.3 ローカルプロジェクトのインポート	8
4.4 プロジェクトのアップロード	9
4.5 プロジェクトを Python ファイルとして保存	14
4.6 Python エディタ 基本構文学習	14
5. Pybit 基本チュートリアル	15
5.1 バッテリー残量計	16
5.2 ヘッドライト	18
5.3 RGB ライト	20
5.4 赤外線反射センサー	21
5.5 超音波センサー	23
5.6 車輪の回転	25
6. Pybit プロジェクトチュートリアル	28
6.1 等加速度直線運動	29
6.2 黒線追従	32
6.3 境界巡回	35
6.4 固定距離追従	38
6.5 超音波障害物回避	42
6.6 追光	46
6.7 Micro:bit 無線制御	49
6.8 Micro:bit 加速度センサー無線制御	52
7. Pybit 拡張チュートリアル	56
7.1 サーボ駆動	57
8. Q&A	59
8.1 Micro:bit にコードをアップロードできません	59
8.2 Micro:bit のドライブレター表示が異常です	59
8.3 コードを再アップロード後もモーターが回転し続ける	59
8.4 モーターとヘッドライトの両方が作動しない	60
8.5 その他の故障	60
9. お問い合わせ	60

1. チュートリアル説明

1.1 Python は、デスクトップやサーバーなどの高性能ハードウェア上で動作する汎用プログラミング言語です。MicroPython は、マイクロコントローラや組み込みデバイス向けに特別に設計された、合理化されたバージョンの Python です。

1.2 このチュートリアルでは、プログラミングに MicroPython を使用します。Micro:bit には専用の NEC 赤外線 MicroPython ライブラリがありません。Pin を使用して NEC 赤外線信号を読み取る場合、読み取り速度が遅すぎて深刻な信号損失が発生します。したがって、このチュートリアルでは NEC 赤外線リモコン制御を実装できません。

1.3 Micro:bit V1 はコアが 1 つしかないため、MicroPython を実行しながら Bluetooth を実行することはできません。最新の Micro:bit V2 はコアが 2 つあり、理論上は MicroPython と Bluetooth を同時に実行できますが、この機能はまだ公式にテスト中です。詳細な情報については、以下を確認してください：

<https://microbit-micropython.readthedocs.io/en/v2-docs/ble.html>

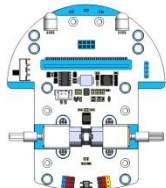
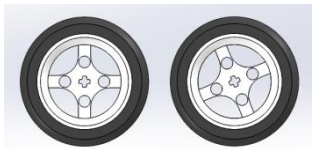
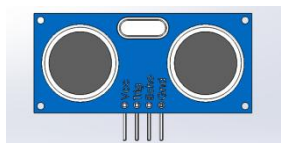
上記の理由により、このチュートリアルでは Bluetooth 制御機能を実装できません。今後、公式のデバッグ情報が利用可能になり次第、チュートリアルを更新する予定です！

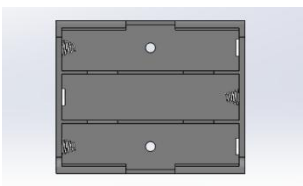
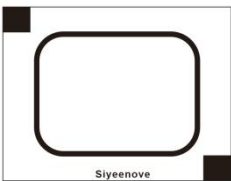
[本チュートリアルの著作権は SIYEENOVE チームに帰属します。SIYEENOVE チームはいつでもチュートリアル内容をアップグレードする権利を留保します。]

2. Pybit 紹介

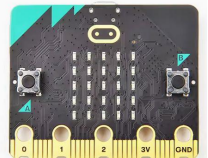
Pybit は、Siyeenove によって開発された、メーカー教育と DIY を中心に設計された micro:bit カーで、非常に便利な取り付けが可能です。超音波モジュール、RGB ライト、赤外線受信センサー、赤外線ライントレースセンサーを統合しています。車体にはレゴ取り付け穴と IO 拡張ポートがあり、さまざまなタイプのモジュールを接続するのに非常に便利です。7 歳以上の子供から大学生、電子工作愛好家まで、誰にでも適しています。

2.1 部品リスト

PCB シャーシ	車輪	超音波センサー
		

単三電池ボックス	IR リモコン	マップ
		
3M 両面テープ		
		

注記: 単三電池と **Micro:bit v2.x.x** メインボードは別途購入する必要があります。

単三電池	Micro:bit - v2.x.x
	

2.2 仕様

使用メインボード: [Micro:bit - v2.x.x](#)

プログラミング方法: [MakeCode](#) および [Python Editor](#)

バッテリーコネクタタイプ: XH2.54 2P, 入力電圧 3--5.5V, デフォルトで単三電池 3 本を直列で使用。

赤外線リモコンタイプ: NEC, リモコン距離 ≥ 5 メートル。

モータータイプ: GA12-N20 マイクロ DC ギアードモーター (120 RPM)。

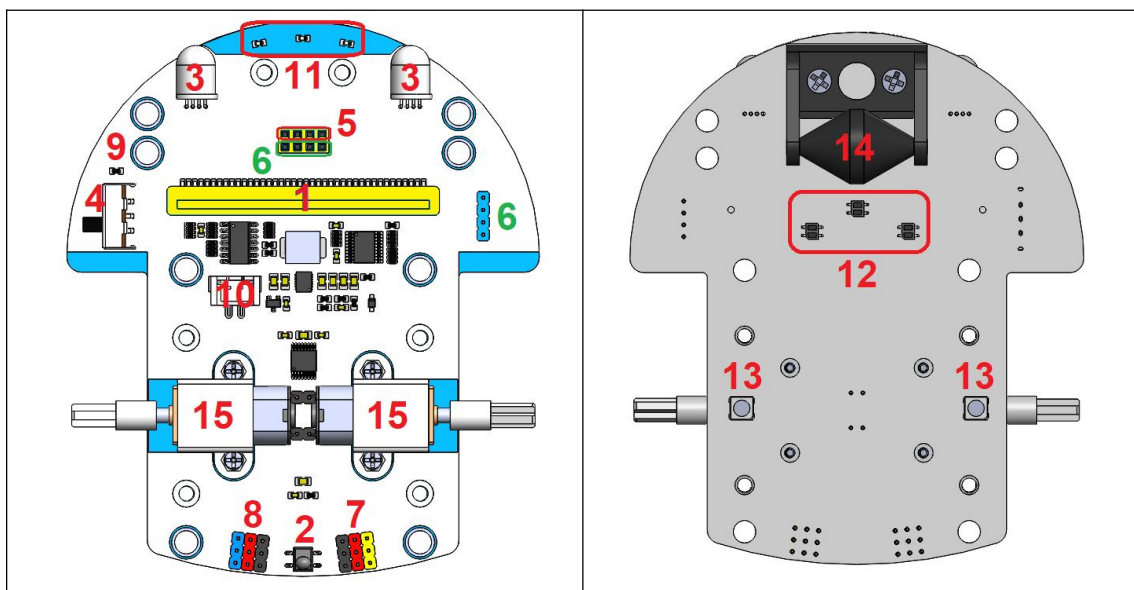
超音波タイプ: HC-SR04 (範囲: 2cm-400cm, 精度: 3mm)。

拡張ポートタイプ: 2.54mm ピッチピンヘッダー。

車輪直径: 53mm

外形寸法 (L*W*H): 117.5*97*70mm

2.3 部品説明



Micro:bit スロット: 1

Micro:bit v2.x.x メインボードをここに挿入します。

赤外線受信センサー: 2

NEC 赤外線通信プロトコル、信号ピンは Micro:bit の P9 ピンに接続されています。

ヘッドライト: 3

8mm RGB LED ライト、左右に分かれており、STC8H1K08 によって制御されます。Micro:bit の I2C ポートから関連するコマンドを STC8H1K08 に送信して制御します。

電源スイッチ: 4

電池ボックスの電源の ON/OFF を制御します。

超音波インターフェース: 5

V=3.3V, G=GND, 外部 HC-SR04 超音波モジュール接続用。信号ピンは Micro:bit の P12 (Echo) と P13 (Trig) ピンに接続されています。

I2C ポート: 6

Micro:bit の I2C 通信インターフェース、SDA=P20, SCL=P19, 他の I2C デバイスと通信できます。

サーボインターフェース: 7

外部サーボ 3 個接続用、90 度、180 度、270 度、360 度サーボの駆動に使用されます。

IO 拡張ポート: 8

他のセンサーモジュールを接続できます。

電源インジケータ: 9

電池タイプ設定後の正常動作時は点灯。電圧が 20%電圧レベル以下になると点滅します。

バッテリーコネクタ: 10

XH2.54 2P, 入力電圧 3--5.5V。

赤外線ライントレースインジケータ: 11

黒線を検出すると点灯し、それ以外は消灯します。

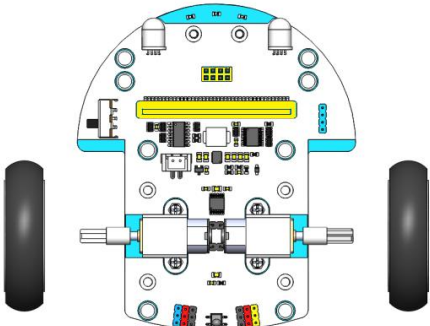
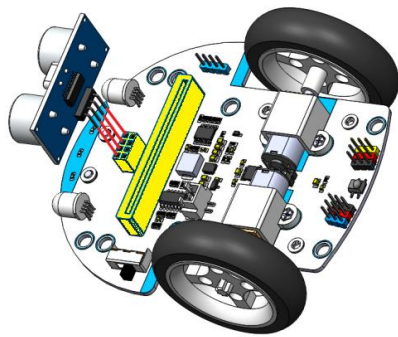
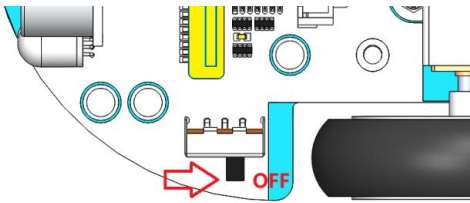
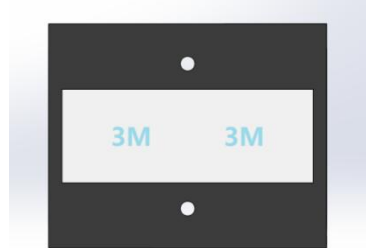
赤外線ライントレースセンサー: 12

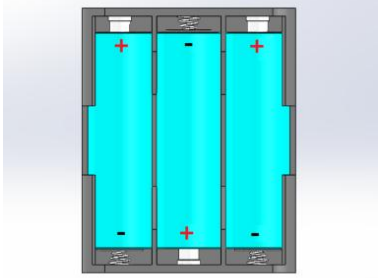
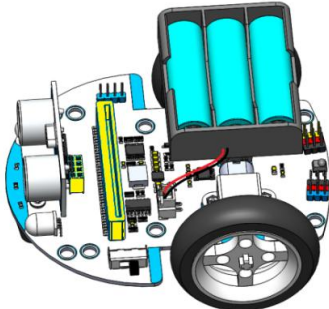
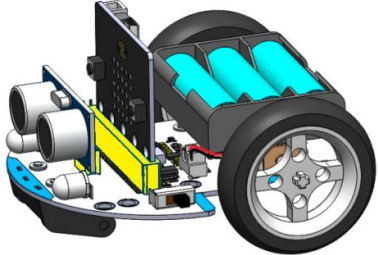
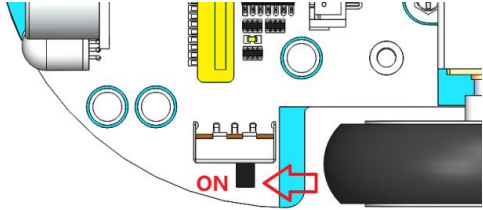
赤外線反射原理により黒白線を検出します。左、中央、右の 3 つのセンサーに分かれています。信号ピンはそれぞれ Micro:bit の P14, P15, P16 ピンに接続されています。

RGB ライト: 13

WS2812 RGB ライト、制御可能な赤、緑、青の原色を発光できます。合計 2 個のライト、番号は RGB1-RGB2。三原色の強度をそれぞれ制御することで、さまざまな色の光を形成できます。信号入力ピンは Micro:bit の P8 ピンに接続されています。

3. Pybit の取り付け

ステップ 1: 車輪の取り付け 	ステップ 2: 超音波モジュールの取り付け 
ステップ 3: 電源オフ (注記: Micro:bit V2 の挿抜時は電源をオフにし、損傷を避けてください。) 	ステップ 4: 電池ボックスの底面中央に 3M 両面テープを貼り付ける 
ステップ 5: 単三電池の取り付け	ステップ 6: 電池ボックスを車体に貼り付ける

	
ステップ 7: Micro:bit V2 ボードの挿入 (注記 ドットマトリックスが前を向く)	ステップ 8: 電源スイッチをオン
	

注記: **Micro:bit** メインボードの挿入または取り外し時は、メインボードの損傷を避けるために電源スイッチをオフにすることを推奨します!

4. Python エディタ 必須知識

Python Editor は、BBC Micro:bit で MicroPython プログラミングを始めるのに最適な方法です。Python Editor は無料で、すべてのプラットフォームのブラウザで動作します。

Google Chrome または Microsoft Edge ブラウザの使用を推奨します。WebUSB は、Web ページから直接 Micro:bit にアクセスできる、最新の開発中の Web 機能です。また、Micro:bit から直接 Python Editor にデータを送信することもできます。Google Chrome と Microsoft Edge ブラウザで動作します。

WebUSB サポート Micro:bit バージョン

最新バージョンの Google Chrome または Microsoft Edge を使用していない場合は、以下のバージョン以上であることを確認してください:

Google Chrome (バージョン 79 以上) for Android, Chrome OS, Linux, macOS, and Windows 10.

Microsoft Edge (バージョン 79 以上) for Android, Chrome OS, Linux, macOS, and Windows 10.

最新の Google Chrome ブラウザをダウンロードするリンク:

<https://www.google.com/chrome/>

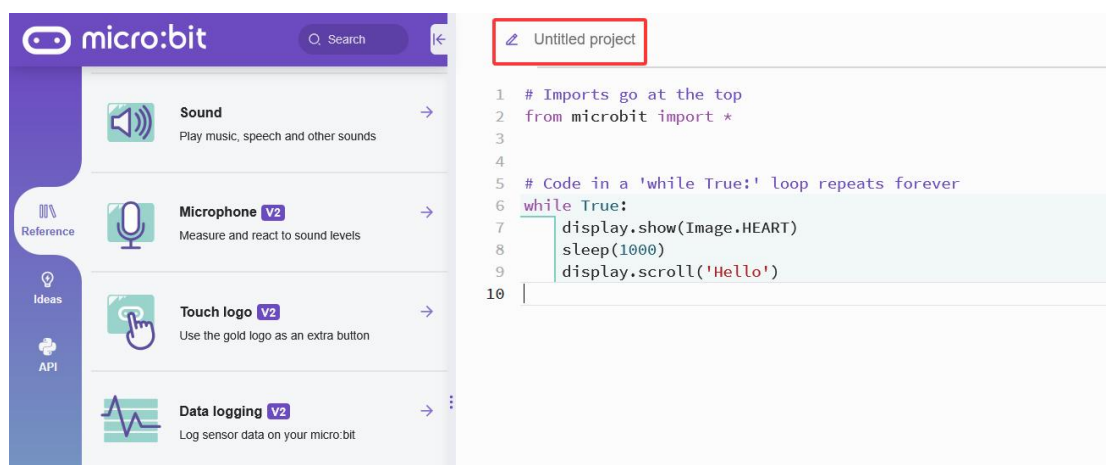
最新の Microsoft Edge をダウンロードするリンク:

<https://www.microsoft.com/en-us/edge/download>

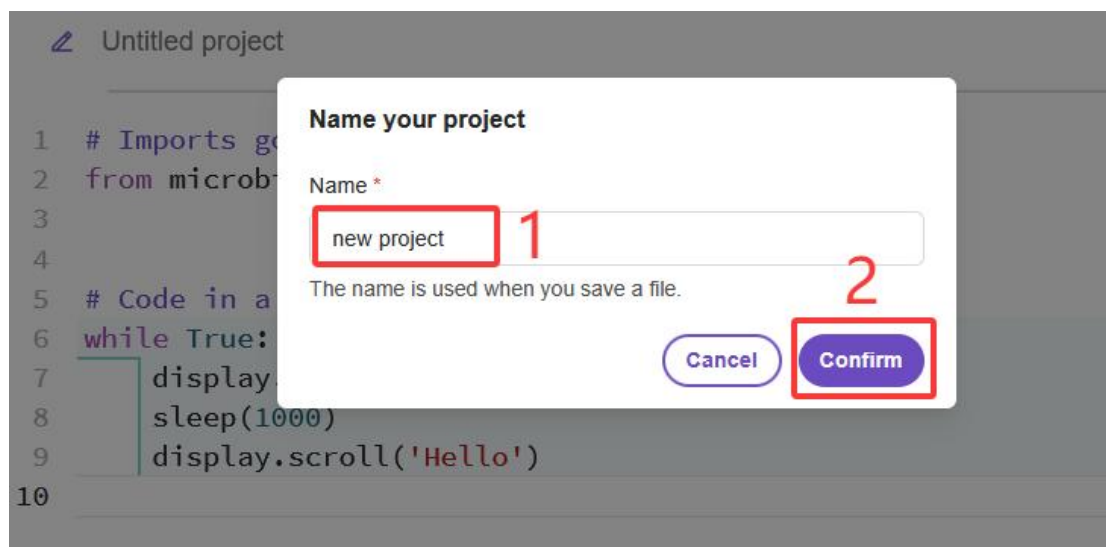
4.1 新規プロジェクトの作成

PC の Google Chrome ブラウザでオンラインプログラミングページを開く:

<https://python.microbit.org/v/3> 赤枠をクリック:

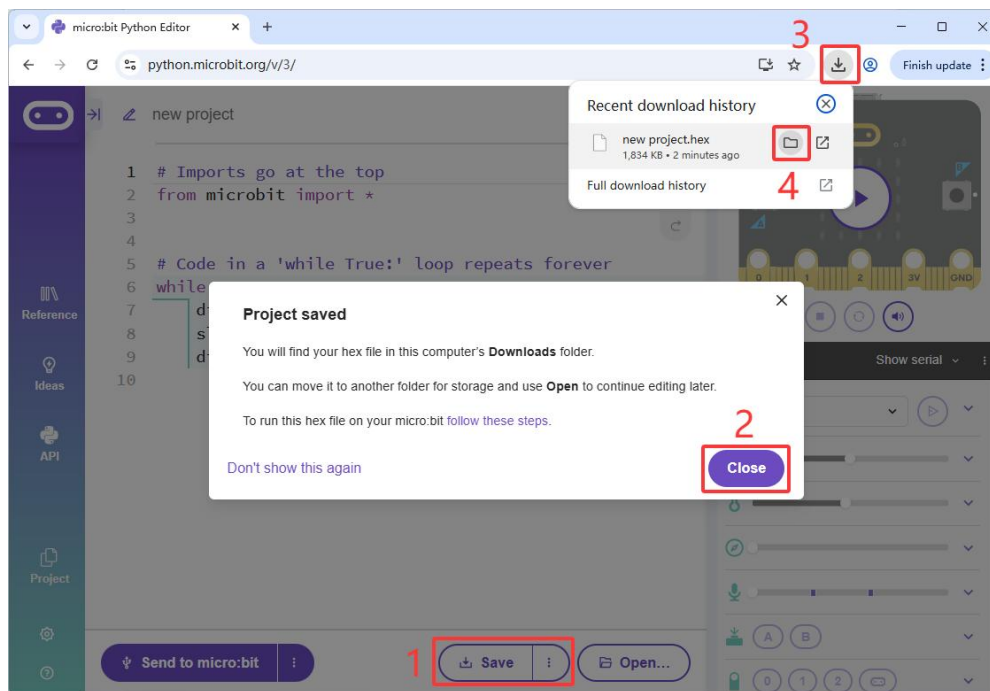


その後、プロジェクト名を入力し、「確認」をクリック:



4.2 プロジェクトのローカル保存

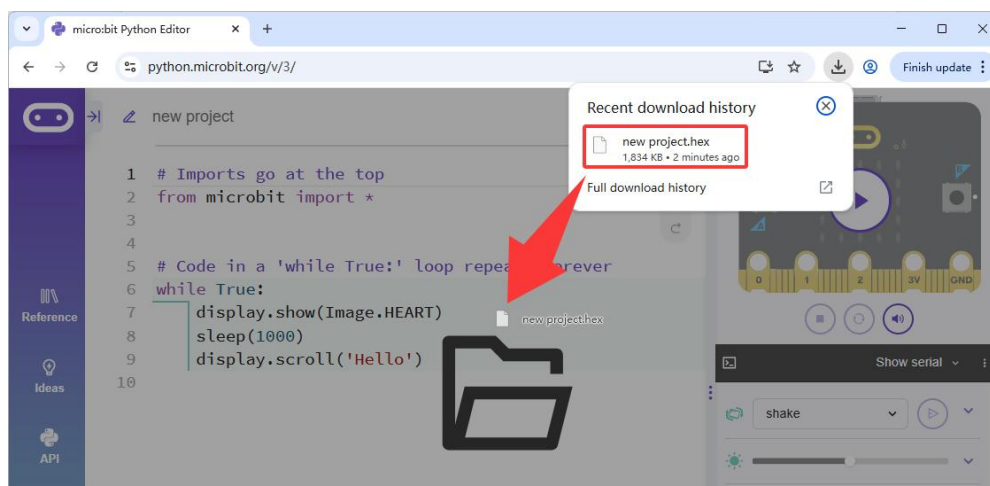
ステップ1「保存」ボタンをクリックし、次にステップ2「閉じる」ボタンをクリックしてプロジェクトをローカルに保存します。その後、ステップ3と4に従って保存された「.hex」ファイルを探します。下記を参照：



注記: このポップアップウィンドウを再表示させたくない場合は、ステップ2で「再表示しない」をクリックしてください！

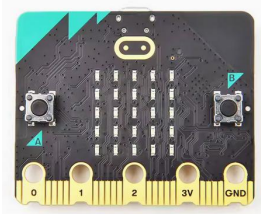
4.3 ローカルプロジェクトのインポート

ローカルの「HEX」プロジェクトファイルを Python Editor の作業領域に直接ドラッグ & ドロップします。下記を参照：

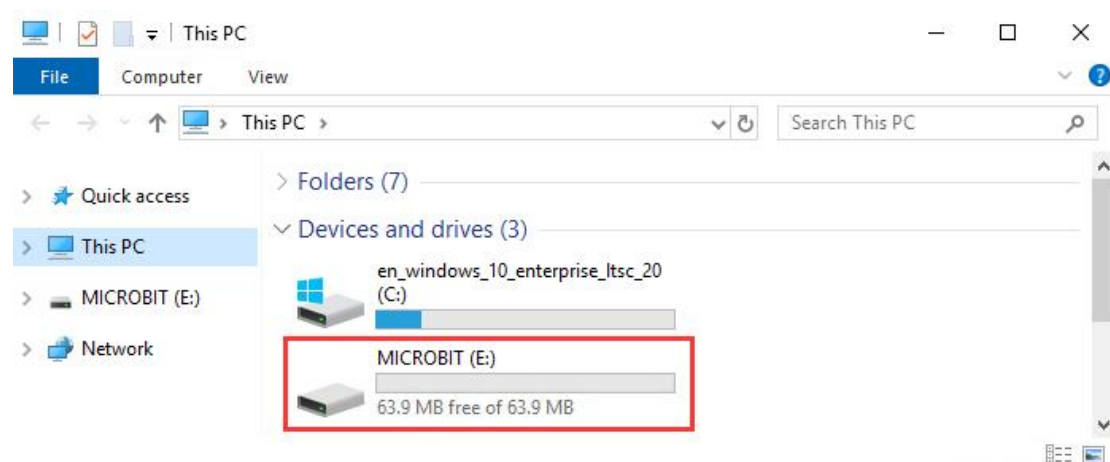
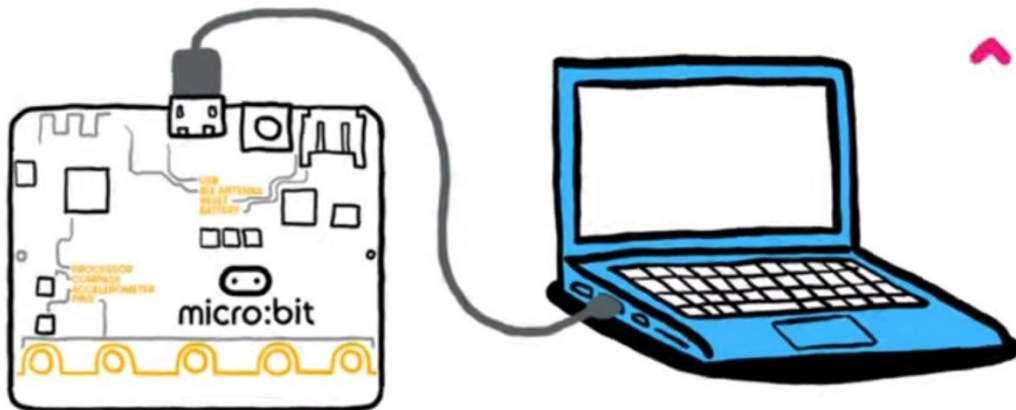


4.4 プロジェクトのアップロード

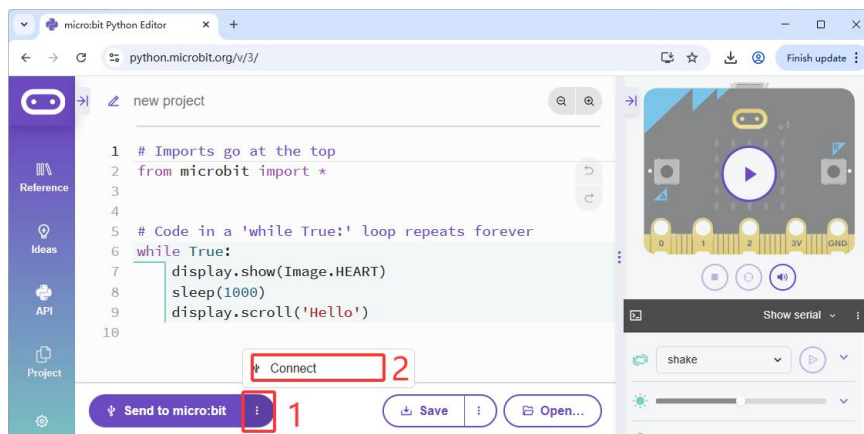
ツール:

PC	Micro:bit v2.x.x	Micro USB ケーブル
		

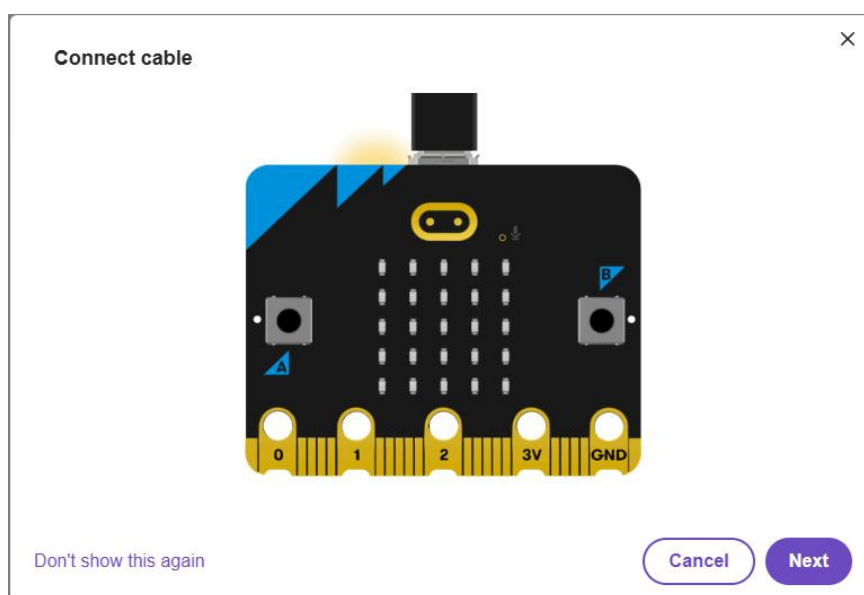
Micro USB ケーブルを使用して Micro:bit メインボードを PC に接続します。PC にストレージドライブが表示されます



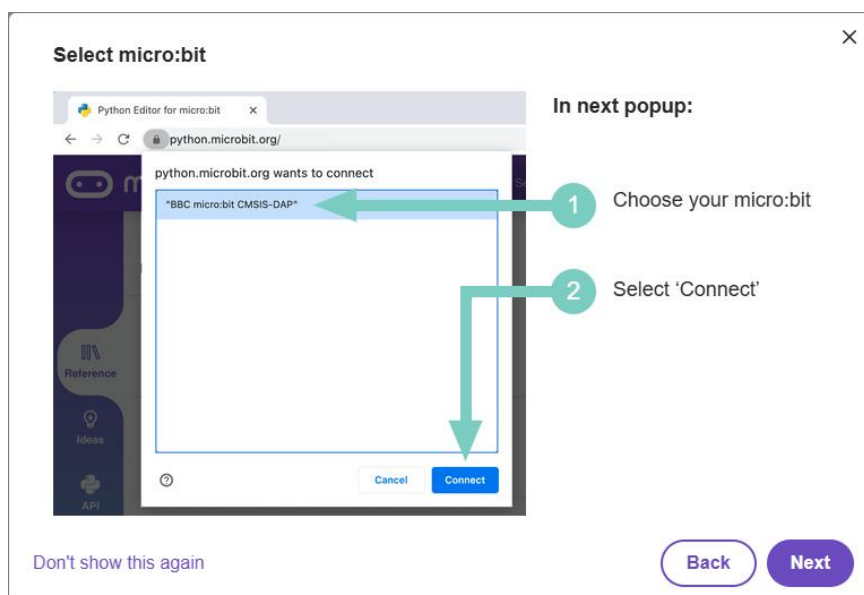
「接続」ボタンをクリック：



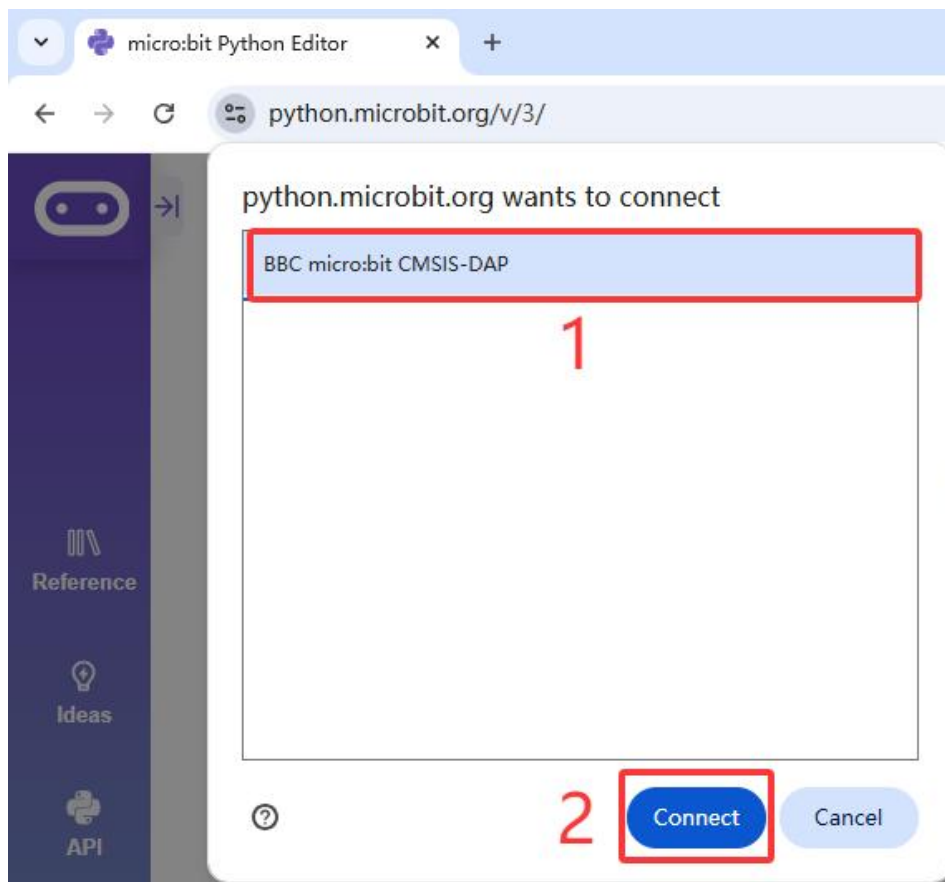
「次へ」をクリック：



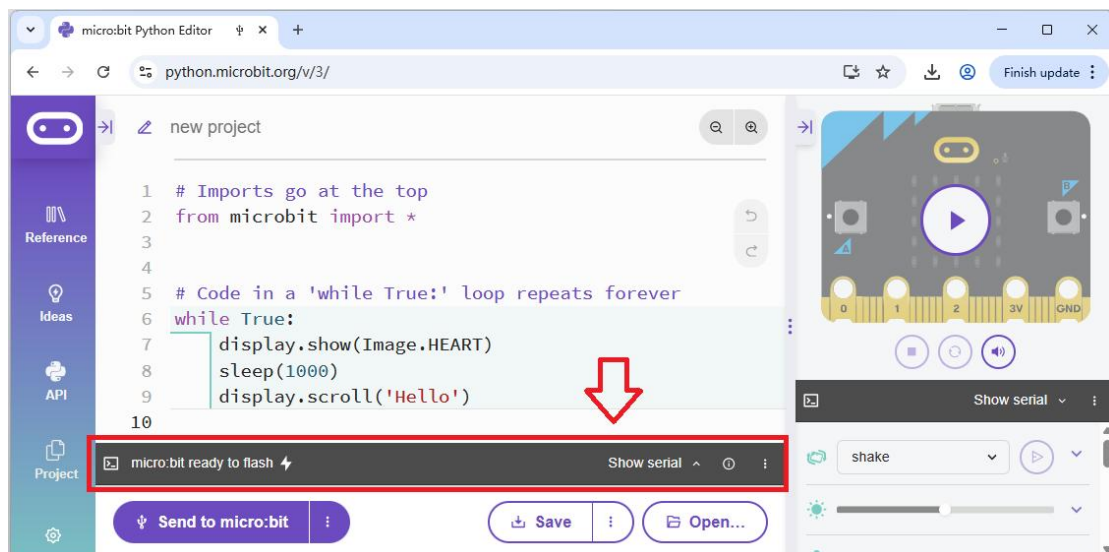
「次へ」をクリック：



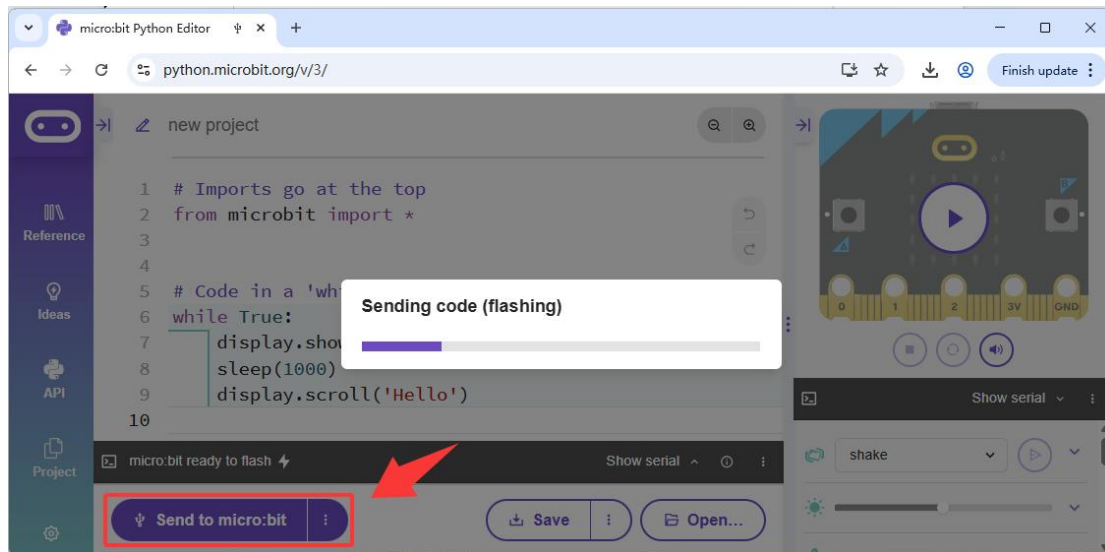
その後、以下の手順に従います：



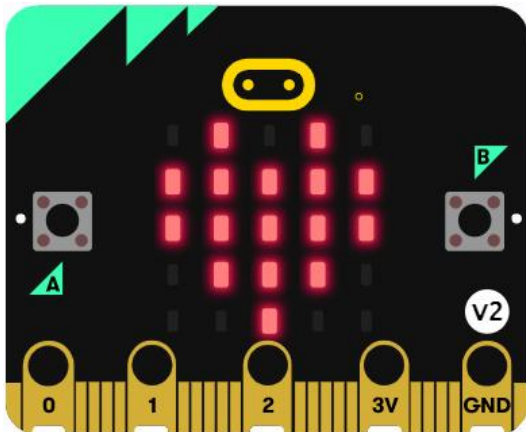
以下の表示が現れたら、Micro:bit と Python Editor の接続が成功したことを示します：



ここで「micro:bit に送信」をクリックして、Python Editor から Micro:bit にコードをアップロードします：

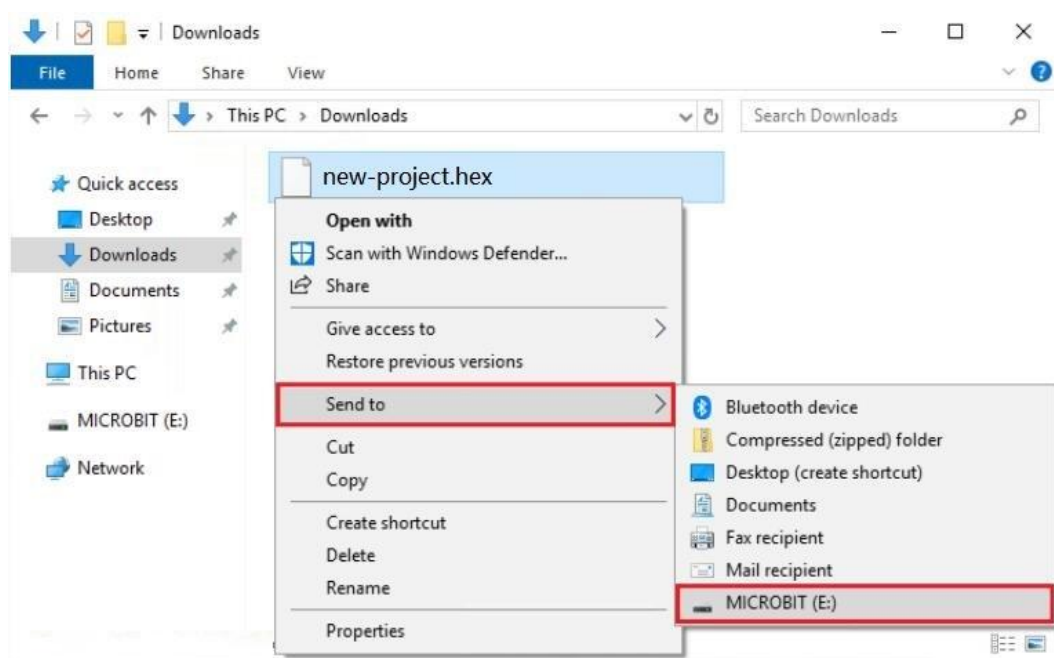


コードがアップロードされると、Micro:bit メインボードのドットマトリックスにハート形が表示され、その後「Hello」がスクロール表示されます：

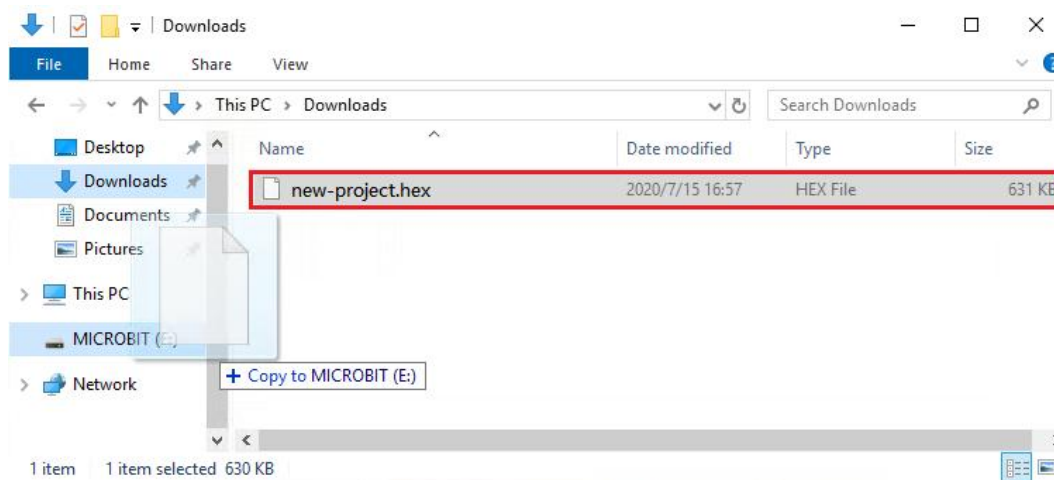


コードをアップロードする他の 2 つの方法

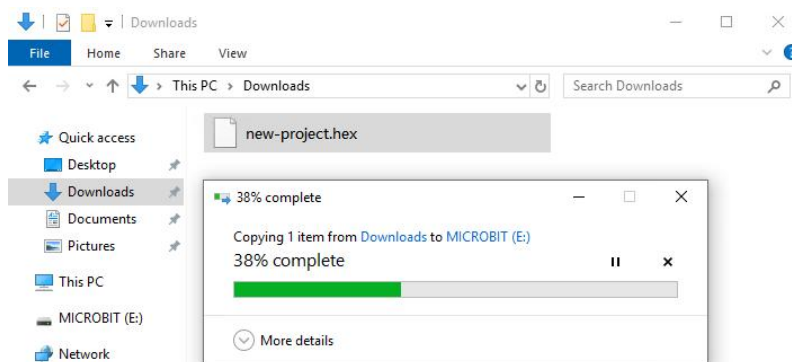
「HEX」ファイルを選択し、右クリックして「HEX」ファイルを Micro:bit メインボードに送信します：



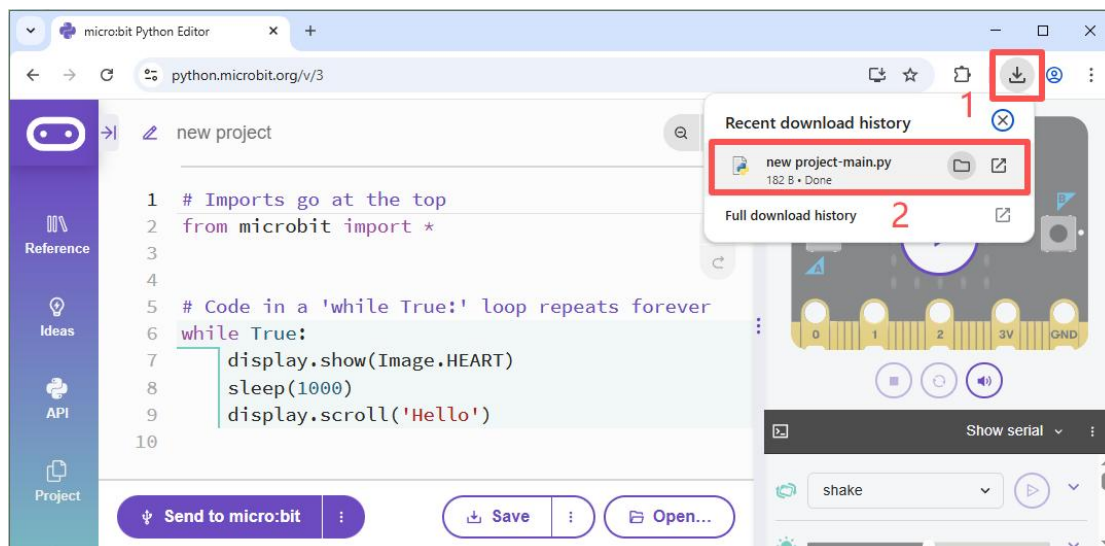
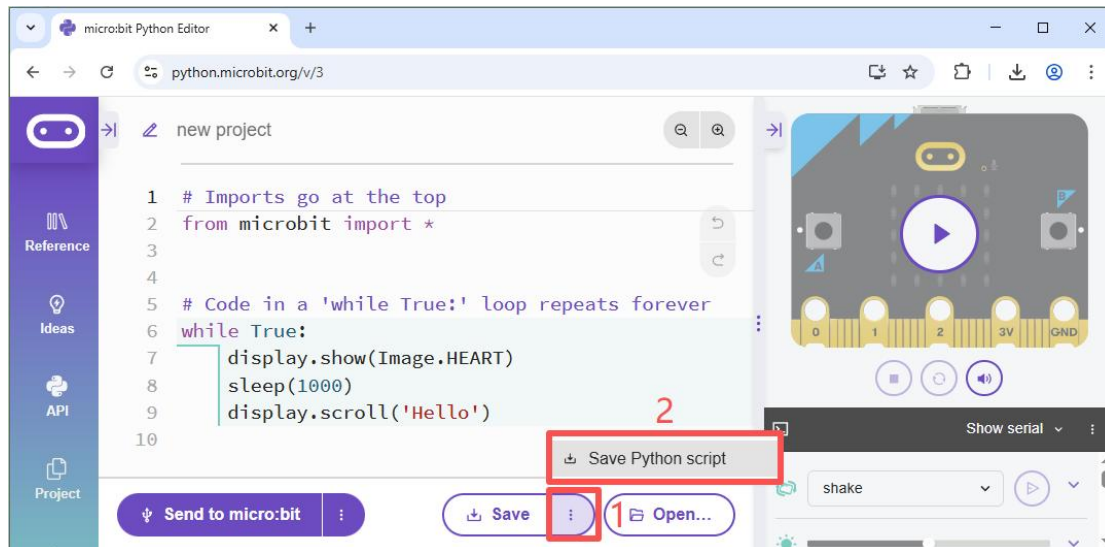
または、直接 Micro:bit メインボードにドラッグ & ドロップします：



以下のインターフェースはコードがアップロード中であることを示します：



4.5 プロジェクトを Python ファイルとして保存



注記: Pythonの「xx.py」ファイルはMicro:bit上で直接実行できません。Micro:bit上で実行できるのはプロジェクトの「xx.hex」ファイルのみです。「xx.py」ファイルは一般的なテキスト編集ソフトウェアで開くことができますが、「xx.hex」ファイルは開けません！

4.6 Python エディタ 基本構文学習

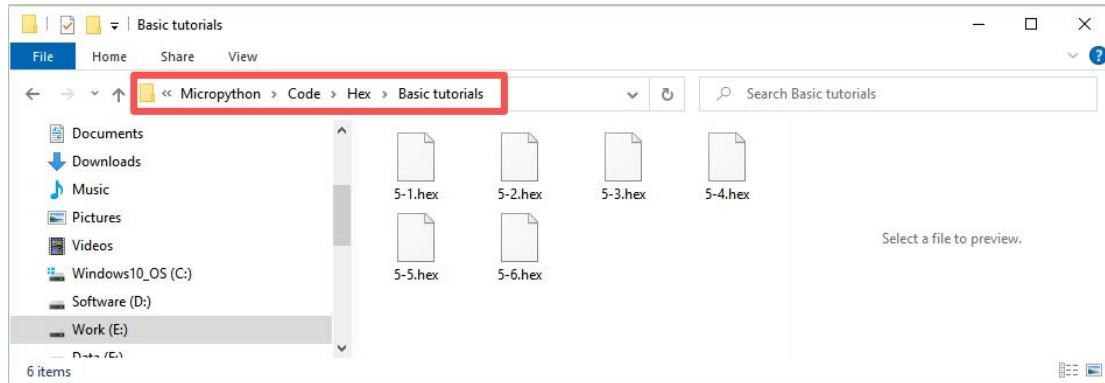
Micro:bitプラットフォームは、公式の Python Editor 使用参考ドキュメントを提供しています。

ユーザーガイド: <https://microbit.org/get-started/user-guide/python-editor/>

MicroPython ドキュメント: <https://microbit-micropython.readthedocs.io/en/v2-docs/>

5. Pybit 基本チュートリアル

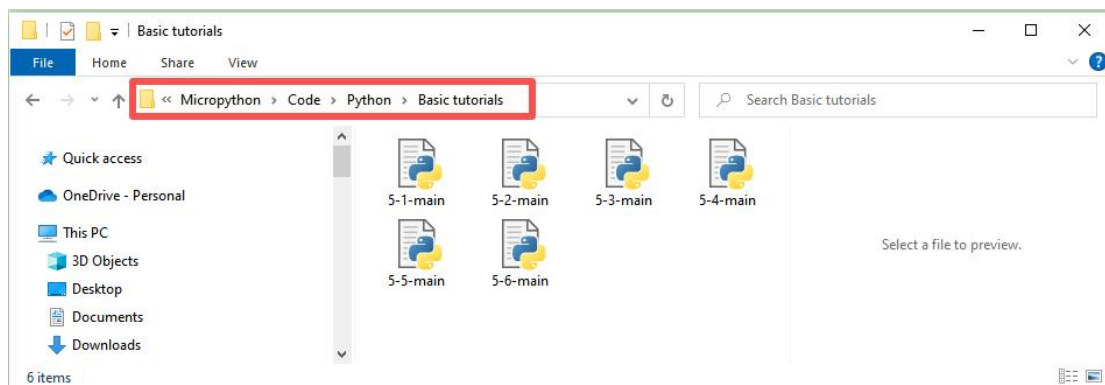
基本チュートリアルのサンプルプロジェクトはすべて、「Micropython -> Code -> Hex -> Basic tutorials」フォルダに保存されています：



特別な注意!!!

以下のチュートリアルでは、提供されているローカルのサンプルプロジェクトをインポートする場合は、セクション 4.3 を参照してください。新規プロジェクトを作成する場合は、セクション 4.1 を参照してください。

基本チュートリアルのサンプル Python ファイルは、「Micropython -> Code -> Python -> Basic tutorials」フォルダに保存されています：

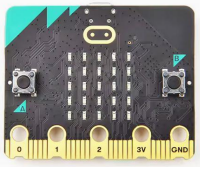
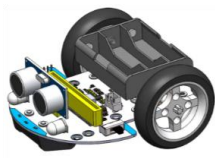


5.1 バッテリー残量計

目的

バッテリーの種類を設定し、バッテリーの電圧レベルを読み取ります。

ツール

Computer	Micro:bit v2.x.x	Micro USB Cable	Pybit
			

プログラミング

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# Type of battery
aaBattery = bytearray([0x01])
lithiumBattery = bytearray([0x02])

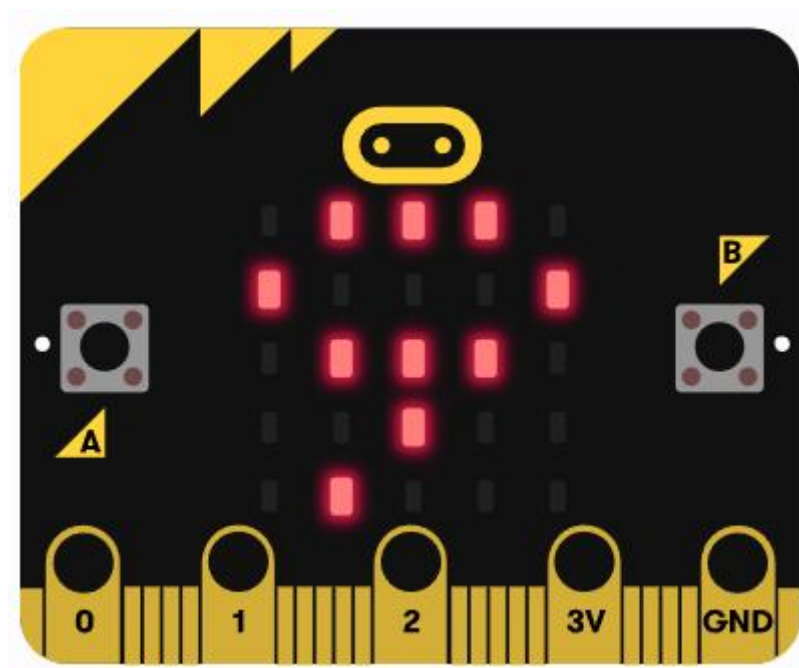
# Initialize the I2C communication interface
i2c.init()

# Code in a 'while True:' loop repeats forever
while True:
    # Read the battery power level
    i2c.write(i2cAddr, aaBattery, True)
    batLevel = i2c.read(i2cAddr, 1)

    sleep(1000)                # Delay: 1000 milliseconds
    display.scroll(batLevel[0]) # Scroll display of battery level
    sleep(3000)                # Delay: 3000 milliseconds
```

結論

Micro:bit のドットマトリックスが読み取った電圧レベルをスクロール表示します。値の範囲は 0〜100%です。また、バッテリー電圧レベルが 20%以下の場合、バッテリーインジケータライトは点滅せずに点灯したままになります。それ以外の場合は点滅します。



考察

電圧レベルが 20%未満の場合に、Micro:bit のブザーで警報を鳴らすにはどうすればよいですか？

よくある問題

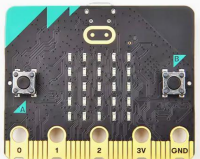
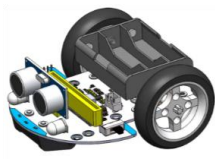
バッテリー電圧レベルを読み取る文は、バッテリーの種類を設定するために、プログラム内で少なくとも1回は実行する必要があります。そうしないと、バッテリー電圧インジケータが正常に動作しません！

5.2 ヘッドライト

目的

Pybit のヘッドライトを点滅させます。

ツール

PC	Micro:bit メインボード	Micro USB ケーブル	Pybit
			

プログラミング

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For head RGB LED
leftLed = 0
rightLed = 1
i2cBuf = bytearray([0x00, 0x00])

# Set the head RGB LED function
# led: 0 is the left led and 1 is the right LED.
# r: red brightness value
# g: green brightness value
# b: blue brightness value
def setHeadRgbLed(led, r, g, b):
    if led == leftLed:
        i2cBuf[0] = 0x07 # red register
        i2cBuf[1] = r    # red brightness value
        i2c.write(i2cAddr, i2cBuf)
        i2cBuf[0] = 0x08 # green register
        i2cBuf[1] = g    # green brightness value
        i2c.write(i2cAddr, i2cBuf)
        i2cBuf[0] = 0x09 # blue register
        i2cBuf[1] = b    # blue brightness value
        i2c.write(i2cAddr, i2cBuf)
    if led == rightLed:
```

```

i2cBuf[0] = 0x0a # red register
i2cBuf[1] = r    # red brightness value
i2c.write(i2cAddr, i2cBuf)
i2cBuf[0] = 0x0b # green register
i2cBuf[1] = g    # green brightness value
i2c.write(i2cAddr, i2cBuf)
i2cBuf[0] = 0x0c # blue register
i2cBuf[1] = b    # blue brightness value
i2c.write(i2cAddr, i2cBuf)

# Code in a 'while True:' loop repeats forever
while True:
    setHeadRgbLed(leftLed, 255, 0, 0) # red
    setHeadRgbLed(rightLed, 255, 0, 0)
    sleep(1000)
    setHeadRgbLed(leftLed, 0, 255, 0) # green
    setHeadRgbLed(rightLed, 0, 255, 0)
    sleep(1000)
    setHeadRgbLed(leftLed, 0, 0, 255) # blue
    setHeadRgbLed(rightLed, 0, 0, 255)
    sleep(1000)

```

結論

Pybit の 2 つのヘッドライトが色を循環します: 赤 -> 1 秒遅延 -> 緑 -> 1 秒遅延 -> 青 -> 1 秒遅延。

考察

ヘッドライトをより速く、より多くの色で表示させるにはどうすればよいですか？

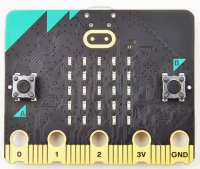
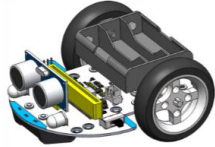
よくある問題

5.3 RGB ライト

目的

Pybit の底面にある 2 つの RGB ライトを点滅させます。

ツール

PC	Micro:bit メインボード	Micro USB ケーブル	Pybit
			

プログラミング

```
# micro:bit V2 only
# Imports go at the top
from microbit import *
import neopixel

np = neopixel.NeoPixel(pin8, 2) # Using pin8, 2 neopixel.
# Code in a 'while True:' loop repeats forever
while True:
    np.fill((255, 0, 0)) # Light all NeoPixels red
    np.show()
    sleep(1000)
    np.fill((0, 255, 0)) # Light all NeoPixels green
    np.show()
    sleep(1000)
    np.fill((0, 0, 255)) # Light all NeoPixels blue
    np.show()
    sleep(1000)
    np.clear()          # Clear NeoPixels

    np[0] = (255, 0, 0) # A NeoPixel bright red
    np[1] = (0, 255, 0) # The other NeoPixel is bright green
    np.show()
    sleep(1000)
    np.clear()          # Clear NeoPixels
```

結論

Pybit の底面にある 2 つの虹色ライトが色を循環します: 赤 -> 1 秒遅延 -> 緑 -> 1 秒遅延 -> 青 -> 1 秒遅延。

考察

虹色ライトをより速く、より多くの色で表示させるにはどうすればよいですか?

よくある問題

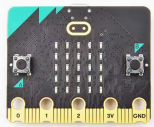
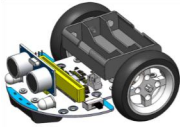
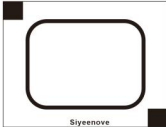
このコードは micro:bit V2 でのみ実行できます。micro:bit V1 ではエラーが報告されます!

5.4 赤外線反射センサー

目的

Pybit の 3 つの赤外線ライントレースセンサーを使用して、黒と白を識別します。

ツール

PC	Micro:bit 主板	Micro USB 线	Pybit	地图
				

プログラミング

```
# Imports go at the top
from microbit import *

channel1 = 0 # left = P14
channel2 = 0 # centre = P15
channel3 = 0 # right = P16

pin14.set_pull(pin14.PULL_UP)
pin15.set_pull(pin15.PULL_UP)
pin16.set_pull(pin16.PULL_UP)

# Code in a 'while True:' loop repeats forever
while True:
    # Read the values of three sensors, 0 or 1.
```



```

# 0: Black is detected.
# 1: White is detected.

channel1 = pin14.read_digital()
channel2 = pin15.read_digital()
channel3 = pin16.read_digital()

# Convert the values of the three sensors into one data.
# Binary data notation:
# 0b bit2 bit1 bit0
# channel1 = bit2, channel2 = bit1, channel3 = bit0.
value = (channel1 << 2) + (channel2 << 1) + channel3

if value == 0b111:
    display.show(Image.NO)
if value == 0b000:
    display.show(Image.YES)
sleep(250)

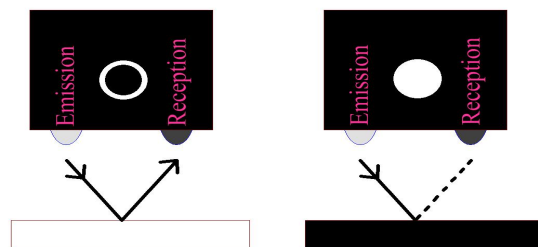
```

結論

表面が粗い黒色物体は赤外線をよく吸収します。センサーが発信した赤外線が黒い線に当たると吸収され、白色の物体に当たると反射されます。

コードをアップロード後、Pybit の 3 つの赤外線センサー全てをマップの黒色

領域に置くと、Micro:bit のドットマトリックスに「✓」が表示されます。逆に、3 つのセンサー全てを白色領域に置くと、「×」が表示されます。



考察

どの赤外線センサーが黒色物体を認識したか、どうやって区別しますか？

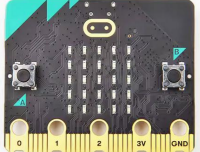
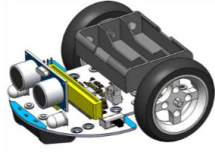
よくある問題

5.5 超音波センサー

目的

Pybit の超音波センサーを使用して距離を測定します。

ツール

PC	Micro:bit メインボード	Micro USB ケーブル	Pybit
			

プログラミング

```
# Imports go at the top
from microbit import *
import utime

# Define the pins of the ultrasonic module.
TRIG_PIN = pin13
ECHO_PIN = pin12

# Initialize the ultrasonic module
def setup_ultrasonic():
    TRIG_PIN.write_digital(0)
    ECHO_PIN.set_pull(ECHO_PIN.NO_PULL)

# Measuring distance (cm)
def measure_distance():
    # A trigger signal of 10µs was sent
    TRIG_PIN.write_digital(1)
    utime.sleep_us(10)
    TRIG_PIN.write_digital(0)

    # Wait for the echo pin to get high
    timeout = 50000 # 50ms timeout
    start_time = utime.ticks_us()
```

```

while ECHO_PIN.read_digital() == 0:
    if utime.ticks_diff(utime.ticks_us(), start_time) > timeout:
        return -1 # timeout

# The echo onset time was recorded
pulse_start = utime.ticks_us()

# Wait for the echo pin to get low
while ECHO_PIN.read_digital() == 1:
    if utime.ticks_diff(utime.ticks_us(), pulse_start) > timeout:
        return -1 # timeout

# The echo end time was recorded
pulse_end = utime.ticks_us()

# Calculate the pulse duration
pulse_duration = utime.ticks_diff(pulse_end, pulse_start)

# Calculating distance (The Speed of Sound
# 340m/s = 0.034cm/μs, Round trip, so divided by 2)
distance = (pulse_duration * 0.034) / 2

return distance

setup_ultrasonic()

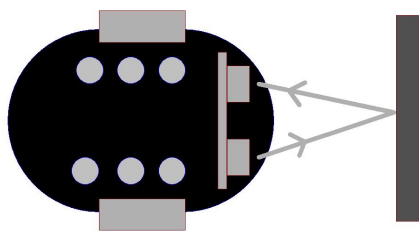
# Code in a 'while True:' loop repeats forever
while True:
    # measuring distance
    dist = measure_distance()

    if dist == -1:    # Measurement timeout
        display.show(Image.NO)
    elif dist > 200:  # Greater than the maximum measurement range
        display.show(Image.ASLEEP)
    else:
        display.scroll(str(int(dist)))    # Displaying distance values

    sleep(1000)

```

結論



Micro:bot のドットマトリックスが測定した障害物までの距離をセンチメートル単位でスクロール表示します。

考察

よくある問題

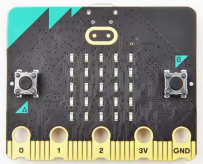
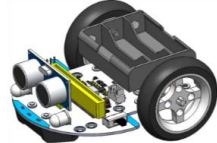
2回の超音波距離測定の間は、50ミリ秒の遅延を設けることを推奨します。これにより、連続する測定波の相互干渉を防止できます。

5.6 車輪の回転

目的

Pybitの車輪を回転させます。

ツール

PC	Micro:bit メインボード	Micro USB ケーブル	Pybit
			

プログラミング

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel = 0
rightwheel = 1
backward = 0
forward = 1
i2cBuf = bytearray([0x00, 0x00])

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0

    if wheel == leftwheel:
        i2cBuf[0] = 0x05 # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06 # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:
```

```
# speed value, 101 is the default required data.
i2cBuf[1] = speed + 101
i2c.write(i2cAddr, i2cBuf)

# Code in a 'while True:' loop repeats forever
while True:

    # CW
    setWheelSpeed(leftwheel, backward, 100)
    setWheelSpeed(rightwheel, forward, 100)
    sleep(1000)

    # stop
    setWheelSpeed(leftwheel, backward, 0)
    setWheelSpeed(rightwheel, forward, 0)
    sleep(1000)

    # CCW
    setWheelSpeed(leftwheel, forward, 100)
    setWheelSpeed(rightwheel, backward, 100)
    sleep(1000)

    # stop
    setWheelSpeed(leftwheel, forward, 0)
    setWheelSpeed(rightwheel, backward, 0)
    sleep(1000)
```

結論

Pybitがその場で左右に旋回します。

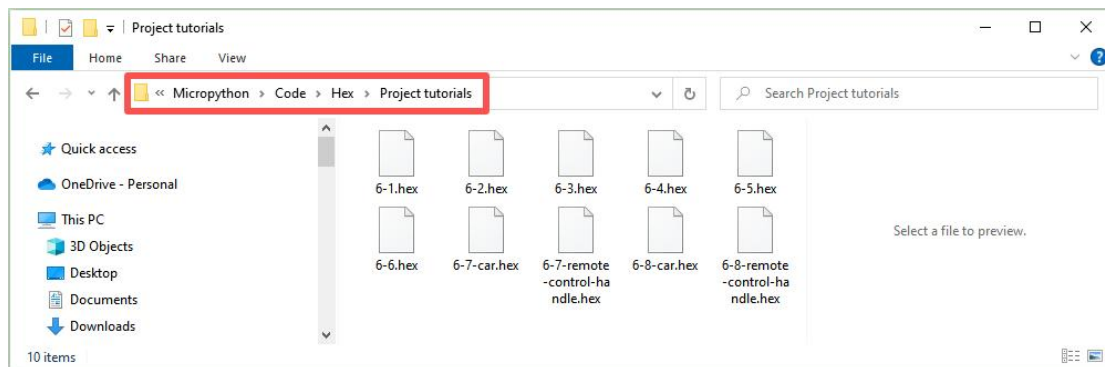
考察

Pybitを前進させるにはどうすればいいですか？

よくある問題

6. Pybit プロジェクトチュートリアル

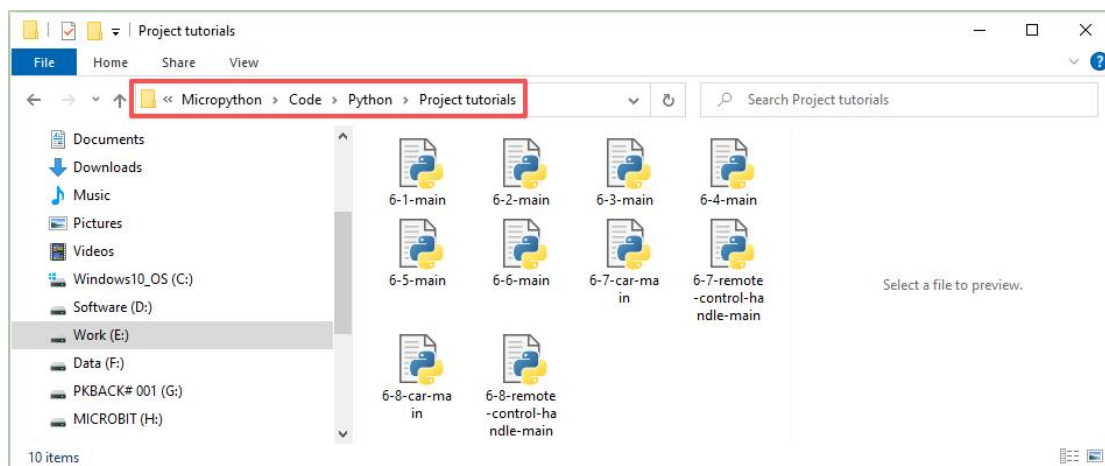
プロジェクトチュートリアルのサンプルプロジェクトはすべて、「Micropython -> Code -> Hex -> Project tutorials」フォルダに保存されています：



特別な注意!!!

以下のチュートリアルでは、提供されているローカルのサンプルプロジェクトをインポートする場合は、セクション 4.3 を参照してください。新規プロジェクトを作成する場合は、セクション 4.1 を参照してください。

プロジェクトチュートリアルのサンプル Python ファイルは、「Micropython -> Code -> Python -> Project tutorials」フォルダに保存されています：



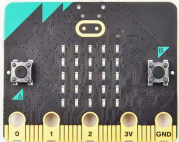
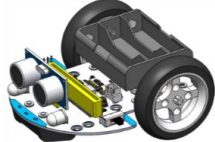
6.1 等加速度直線運動

目的

Pybit のモーターは非常に高出力なため、急激な始動加速度がかかると前部キャスターが浮き上がる可能性があります。

本案例では、均一な加速により滑らかな発進を実現します。

ツール

PC	Micro:bit v2.x.x	Micro USB 線	Pybit
			

プログラミング

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel = 0
rightwheel = 1
backward = 0
forward = 1
i2cBuf = bytearray([0x00, 0x00])

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0
    if wheel == leftwheel:
```

```

i2cBuf[0] = 0x05 # left wheel register
if direction == forward:
    # speed value, 101 is the default required data.
    i2cBuf[1] = speed + 101
elif direction == backward:
    i2cBuf[1] = speed
i2c.write(i2cAddr, i2cBuf)

if wheel == rightwheel:
    i2cBuf[0] = 0x06 # right wheel register
    if direction == forward:
        i2cBuf[1] = speed
    elif direction == backward:
        # speed value, 101 is the default required data.
        i2cBuf[1] = speed + 101
    i2c.write(i2cAddr, i2cBuf)

# Code in a 'while True:' loop repeats forever
while True:
    if speed > 100:
        speed = 0
    # Run forward
    setWheelSpeed(leftwheel, forward, speed)
    setWheelSpeed(rightwheel, forward, speed)

    sleep(30)
    speed = speed + 1

```

結論

車両は循環動作で均一な発進を実現し、速度が速すぎることによる前輪の浮き上がりを防止します。

考察

車両を均一加速後に均一減速させるにはどうプログラムすればよいですか？

よくあるご質問

出荷時に補正値が設定されていない場合、モーターが回転しない可能性があります。その場合は補正値を再設定すれば問題が解決します。

車体内部では 2 つのタイヤの速度を補正可能であり、これにより車体を直進走行させることができます。この補正值は車体の EEPROM に永続的に保存され、出荷時のデフォルト補正值は 0 に設定されています。以下のコードを使用して補正值を再設定できます：

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a
i2cBuf = bytearray([0x00, 0x00])

# For wheel
leftwheel = 0
rightwheel = 1

# Calibrate the wheel speed function
# leftWheelOffset: 0--10
# rightWheelOffset: 0--10
def wheelsAdjustment(leftWheelOffset, rightWheelOffset):
    # The limit variable is between 0 and 10.
    leftWheelOffset = max(0, min(10, leftWheelOffset))
    rightWheelOffset = max(0, min(10, rightWheelOffset))

    i2cBuf[0] = 0x03 # Left wheel offset register
    i2cBuf[1] = leftWheelOffset
    i2c.write(i2cAddr, i2cBuf)
    i2cBuf[0] = 0x04 # Right wheel offset register
    i2cBuf[1] = rightWheelOffset
    i2c.write(i2cAddr, i2cBuf)

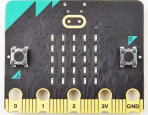
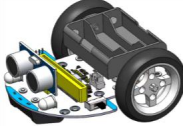
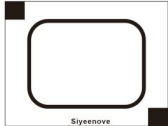
wheelsAdjustment(0, 0)
while True:
    None
```

6.2 黒線追従

目的

Pybit を黒線追従させます。

ツール

PC	Micro:bit メインボード	Micro USB ケーブル	Pybit	マップ
				

プログラミング

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel = 0
rightwheel = 1
backward = 0
forward = 1
i2cBuf = bytearray([0x00, 0x00])

# For line Tracking Sensor
channel1 = 0 # left = P14
channel2 = 0 # centre = P15
channel3 = 0 # right = P16
pin14.set_pull(pin14.PULL_UP)
pin15.set_pull(pin15.PULL_UP)
pin16.set_pull(pin16.PULL_UP)

display.show(Image.HAPPY)
sleep(400)

# Set the wheel speed function
```

```
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0

    if wheel == lefthewheel:
        i2cBuf[0] = 0x05 # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == righthewheel:
        i2cBuf[0] = 0x06 # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        i2c.write(i2cAddr, i2cBuf)

# Code in a 'while True:' loop repeats forever
while True:
    # Read the values of three sensors, 0 or 1.
    # 0: Black is detected.
    # 1: White is detected.
    channel1 = pin14.read_digital()
    channel2 = pin15.read_digital()
    channel3 = pin16.read_digital()

    # Convert the values of the three sensors into one data.
    # Binary data notation:
    # 0b bit2 bit1 bit0
    # channel1 = bit2, channel2 = bit1, channel3 = bit0.
    value = (channel1 << 2) + (channel2 << 1) + channel3
```

```

if value == 0b100: # Slow right turn
    setWheelSpeed(leftwheel, forward, 25)
    setWheelSpeed(rightwheel, forward, 0)

if value == 0b001: # Slow left turn
    setWheelSpeed(leftwheel, forward, 0)
    setWheelSpeed(rightwheel, forward, 25)

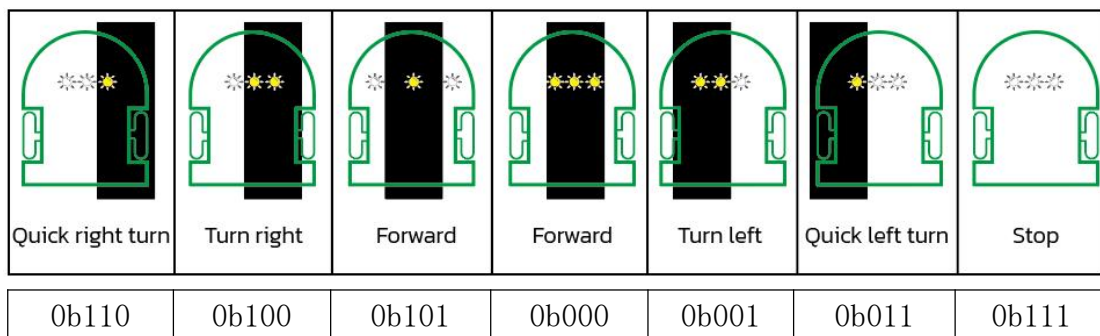
if value == 0b110: # Quick right turn
    setWheelSpeed(leftwheel, forward, 40)
    setWheelSpeed(rightwheel, forward, 0)

if value == 0b011: # Quick left turn
    setWheelSpeed(leftwheel, forward, 0)
    setWheelSpeed(rightwheel, forward, 40)

if value == 0b101 or value == 0b000: # Forward
    setWheelSpeed(leftwheel, forward, 20)
    setWheelSpeed(rightwheel, forward, 20)

if value == 0b111: # Stop
    setWheelSpeed(leftwheel, forward, 0)
    setWheelSpeed(rightwheel, forward, 0)

```

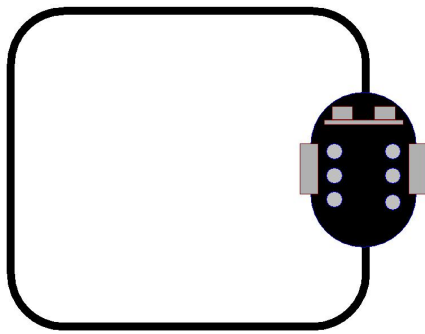


ラインを検出していません (1)。



黒線を検出しました (0)。

結論



Pybit をマップの黒線の上に置くと、Pybit が黒線に沿って走行します。

考察

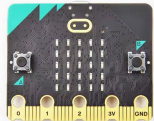
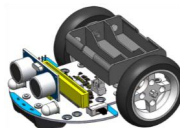
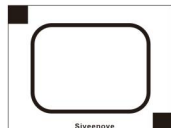
よくある問題

6.3 境界巡回

目的

Pybit を限定区域で巡回させます。

ツール

PC	Micro:bit メインボード	Micro USB ケーブル	Pybit	マップ
				

プログラミング

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For wheel
```

```

leftwheel  = 0
rightwheel = 1
backward   = 0
forward    = 1
i2cBuf = bytearray([0x00, 0x00])

# For line Tracking Sensor
channel1 = 0 # left  = P14
channel2 = 0 # centre = P15
channel3 = 0 # right = P16
pin14.set_pull(pin14.PULL_UP)
pin15.set_pull(pin15.PULL_UP)
pin16.set_pull(pin16.PULL_UP)

display.show(Image.HAPPY)
sleep(400)

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0
    if wheel == leftwheel:
        i2cBuf[0] = 0x05 # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06 # right wheel register
        if direction == forward:

```



```

        i2cBuf[1] = speed
    elif direction == backward:
        # speed value, 101 is the default required data.
        i2cBuf[1] = speed + 101
    i2c.write(i2cAddr, i2cBuf)

# Code in a 'while True:' loop repeats forever
while True:
    # Read the values of three sensors, 0 or 1.
    # 0: Black is detected.
    # 1: White is detected.
    channel1 = pin14.read_digital()
    channel2 = pin15.read_digital()
    channel3 = pin16.read_digital()

    # Convert the values of the three sensors into one data.
    # Binary data notation:
    # 0b bit2 bit1 bit0
    # channel1 = bit2, channel2 = bit1, channel3 = bit0.
    value = (channel1 << 2) + (channel2 << 1) + channel3

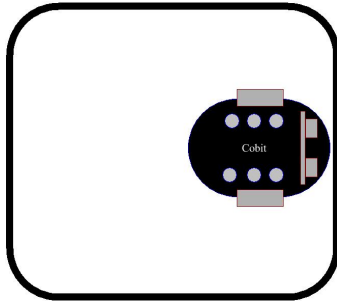
    if value == 0b111:    # Forward
        setWheelSpeed(leftwheel, forward, 40)
        setWheelSpeed(rightwheel, forward, 40)
    else:
        # Stop
        setWheelSpeed(leftwheel, forward, 0)
        setWheelSpeed(rightwheel, forward, 0)
        # Backward
        setWheelSpeed(leftwheel, backward, 40)
        setWheelSpeed(rightwheel, backward, 40)
        sleep(200)

    if random.randint(0, 1) == 0:
        # Left turn
        setWheelSpeed(leftwheel, forward, 0)
        setWheelSpeed(rightwheel, backward, 40)
        sleep(300)
    else:

```

```
# Right turn
setWheelSpeed(leftwheel, backward, 40)
setWheelSpeed(rightwheel, forward, 0)
sleep(300)
```

結論



Pybit は黒い線で囲まれた枠内を継続して走行します。

考察

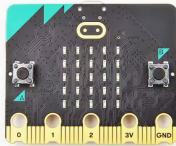
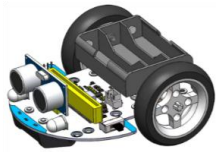
よくある問題

6.4 固定距離追従

目的

あなたの Pybit を、手と一定の距離を保つように動的に動作させます。

ツール

PC	Micro:bit メインボード	Micro USB ケーブル	Pybit
			

プログラミング

```
# Imports go at the top
from microbit import *
import utime

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel = 0
rightwheel = 1
backward = 0
forward = 1
i2cBuf = bytearray([0x00, 0x00])

# Define the pins of the ultrasonic module.
TRIG_PIN = pin13
ECHO_PIN = pin12

# Initialize the ultrasonic module
def setup_ultrasonic():
    TRIG_PIN.write_digital(0)
    ECHO_PIN.set_pull(ECHO_PIN.NO_PULL)
# Measuring distance (cm)
def measure_distance():
    # A trigger signal of 10µs was sent
    TRIG_PIN.write_digital(1)
    utime.sleep_us(10)
    TRIG_PIN.write_digital(0)

    # Wait for the echo pin to get high
    timeout = 50000 # 50ms timeout
    start_time = utime.ticks_us()
    while ECHO_PIN.read_digital() == 0:
        if utime.ticks_diff(utime.ticks_us(), start_time) > timeout:
            return -1 # timeout
```

```

# The echo onset time was recorded
pulse_start = utime.ticks_us()

# Wait for the echo pin to get low
while ECHO_PIN.read_digital() == 1:
    if utime.ticks_diff(utime.ticks_us(), pulse_start) > timeout:
        return -1 # timeout

# The echo end time was recorded
pulse_end = utime.ticks_us()

# Calculate the pulse duration
pulse_duration = utime.ticks_diff(pulse_end, pulse_start)

# Calculating distance (The Speed of Sound
# 340m/s = 0.034cm/μs, Round trip, so divided by 2)
distance = (pulse_duration * 0.034) / 2

return distance

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0

    if wheel == leftrightwheel:
        i2cBuf[0] = 0x05 # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

```

```

if wheel == rightwheel:
    i2cBuf[0] = 0x06 # right wheel register
    if direction == forward:
        i2cBuf[1] = speed
    elif direction == backward:
        # speed value, 101 is the default required data.
        i2cBuf[1] = speed + 101
    i2c.write(i2cAddr, i2cBuf)

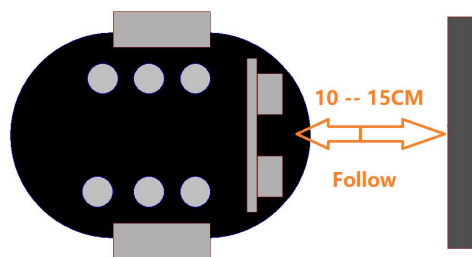
setup_ultrasonic()
display.show(Image.HAPPY)
sleep(400)

# Code in a 'while True:' loop repeats forever
while True:
    # measuring distance
    dist = measure_distance()

    if dist > 10 and dist < 15:
        # Stop
        setWheelSpeed(leftwheel, forward, 0)
        setWheelSpeed(rightwheel, forward, 0)
    elif dist < 10:
        # Backward
        setWheelSpeed(leftwheel, backward, 50)
        setWheelSpeed(rightwheel, backward, 50)
    else:
        # Forward
        setWheelSpeed(leftwheel, forward, 50)
        setWheelSpeed(rightwheel, forward, 50)

```

結論



車体は手から距離が離れすぎている場合は前進して接近し、近すぎる場合は後退して距離を取ります。適切な距離になると停止します。

考察

よくある問題

Q: Pybit を使用中、本来正常に走行していたのに、超音波センサーを接続した後で車両が故障し、走行不能になりました。

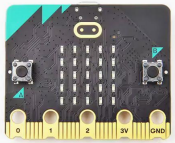
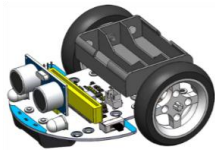
A: 超音波センサーの接続ポートをご確認ください。I2C インターフェースではなく、SONAR 専用ポートに接続する必要があります。

6.5 超音波障害物回避

目的

Pybit を自動障害物回避走行させる方法

ツール

PC	Micro:bit メインボード	Micro USB ケーブル	Pybit
			

プログラミング

```
# Imports go at the top
from microbit import *
import utime
import random

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel = 0
rightwheel = 1
backward = 0
forward = 1
i2cBuf = bytearray([0x00, 0x00])

# Define the pins of the ultrasonic module.
TRIG_PIN = pin13
ECHO_PIN = pin12

# Initialize the ultrasonic module
def setup_ultrasonic():
    TRIG_PIN.write_digital(0)
    ECHO_PIN.set_pull(ECHO_PIN.NO_PULL)
# Measuring distance (cm)
def measure_distance():
    # A trigger signal of 10µs was sent
    TRIG_PIN.write_digital(1)
    utime.sleep_us(10)
    TRIG_PIN.write_digital(0)

    # Wait for the echo pin to get high
    timeout = 50000 # 50ms timeout
    start_time = utime.ticks_us()
    while ECHO_PIN.read_digital() == 0:
        if utime.ticks_diff(utime.ticks_us(), start_time) > timeout:
            return -1 # timeout
```

```

# The echo onset time was recorded
pulse_start = utime.ticks_us()

# Wait for the echo pin to get low
while ECHO_PIN.read_digital() == 1:
    if utime.ticks_diff(utime.ticks_us(), pulse_start) > timeout:
        return -1 # timeout

# The echo end time was recorded
pulse_end = utime.ticks_us()

# Calculate the pulse duration
pulse_duration = utime.ticks_diff(pulse_end, pulse_start)

# Calculating distance (The Speed of Sound
# 340m/s = 0.034cm/μs, Round trip, so divided by 2)
distance = (pulse_duration * 0.034) / 2

return distance

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0

    if wheel == leftrightwheel:
        i2cBuf[0] = 0x05 # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

```



```

if wheel == rightwheel:
    i2cBuf[0] = 0x06 # right wheel register
    if direction == forward:
        i2cBuf[1] = speed
    elif direction == backward:
        # speed value, 101 is the default required data.
        i2cBuf[1] = speed + 101
    i2c.write(i2cAddr, i2cBuf)

setup_ultrasonic()
display.show(Image.HAPPY)
sleep(400)

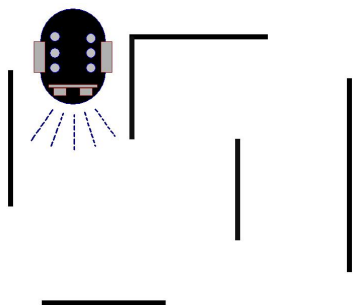
# Code in a 'while True:' loop repeats forever
while True:
    # measuring distance
    dist = measure_distance()
    sleep(200) # Necessary delay

    if dist > 10 and dist < 15:
        # Stop
        setWheelSpeed(leftwheel, forward, 0)
        setWheelSpeed(rightwheel, forward, 0)
        if random.randint(0, 1) == 0:
            # Turn left
            setWheelSpeed(leftwheel, backward, 50)
            setWheelSpeed(rightwheel, forward, 50)
        else:
            # Turn right
            setWheelSpeed(leftwheel, forward, 50)
            setWheelSpeed(rightwheel, backward, 50)
        sleep(200)

    else:
        # Forward
        setWheelSpeed(leftwheel, forward, 100)
        setWheelSpeed(rightwheel, forward, 100)

```

結論



車両は全速前進し、15cm 以内に障害物を検知した場合、ランダムに左または右に回転し、その後も前進を続けます。

考察

Pybit の障害物回避距離を調整するにはどうすればよいですか？

よくある問題

Q: Pybit を使用する際、車体は正常に走行していましたが、超音波センサーを接続した後、故障して走行できなくなりました。

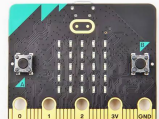
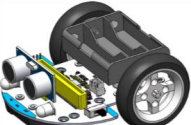
A: 超音波センサーの接続ポートをご確認ください。I2C インターフェースではなく、SONAR 専用ポートに接続する必要があります。

6.6 追光

目的

micro:bitメインボードのドットマトリックスディスプレイが備える光強度感知機能を利用し、Pybitに暗所で光源を自動追従させる動作を実現します。

ツール

PC	Micro:bit メインボード	Micro USB ケーブル	Pybit	懐中電灯
				

プログラミング

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel = 0
rightwheel = 1
backward = 0
forward = 1
i2cBuf = bytearray([0x00, 0x00])

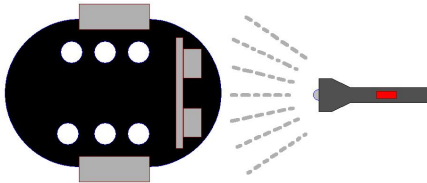
# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0
    if wheel == leftwheel:
        i2cBuf[0] = 0x05 # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06 # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
```

```
i2c.write(i2cAddr, i2cBuf)
```

```
# Code in a 'while True:' loop repeats forever
while True:
    if display.read_light_level() < 50:
        # Turn left at place
        setWheelSpeed(leftwheel, backward, 50)
        setWheelSpeed(rightwheel, forward, 50)
    else:
        # Forward
        setWheelSpeed(leftwheel, forward, 100)
        setWheelSpeed(rightwheel, forward, 100)
```

結論



懐中電灯でMicro:bitメインボードのドットマトリックスを照らすと、光源レベルが50未満を検出した場合、車体はその場で回転し、光源レベルが50以上を検出した場合、車体は全速力で前進します。

考察

なぜ日中環境ではこの機能が実現できない可能性があるのですか？

よくある問題

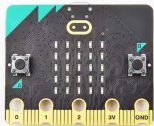
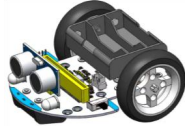
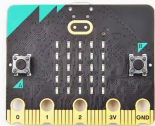
1. この授業は暗所環境で行う必要があり、日中のような明るい環境ではこの機能を実現できません。
2. ご自身で実験を拡張する際は、micro:bitのドットマトリックス機能を一切使用できません。そうしないと、授業で意図した効果が得られません。

6.7 Micro:bit 無線制御

目的

- ▶ 別の Micro:bit メインボードをリモコンとして使用し、Pybit をワイヤレスで操作する。
- ▶ 両方のメインボードにプログラムする必要がある。

ツール

PC	Micro:bit メインボード	Micro USB ケーブル	Pybit	Micro:bit メインボード
				

プログラミング

リモコンのプログラミング:

```
# Imports go at the top
from microbit import *
import radio

# Display image
display.show(Image.HAPPY)
sleep(400)

# Configure the radio
radio.config(group=1, power=3)
radio.on()

# Code in a 'while True:' loop repeats forever
while True:
    if button_a.was_pressed():
        radio.send('1')

    if button_b.was_pressed():
        radio.send('2')

    if button_a.is_pressed() and button_b.is_pressed():
        radio.send('3')
```

Pybit のプログラミング:

```

# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a

# For wheel
leftwheel = 0
rightwheel = 1
backward = 0
forward = 1
i2cBuf = bytearray([0x00, 0x00])

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0

    if wheel == leftwheel:
        i2cBuf[0] = 0x05 # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06 # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:

```

```

        # speed value, 101 is the default required data.
        i2cBuf[1] = speed + 101
        i2c.write(i2cAddr, i2cBuf)

# Display image
display.show(Image.HAPPY)
sleep(400)

# Configure the radio
radio.config(group=1)
radio.on()

# Code in a 'while True:' loop repeats forever
while True:
    message = radio.receive()
    if message == '1':
        # Turn left at place
        setWheelSpeed(leftwheel, backward, 60)
        setWheelSpeed(rightwheel, forward, 60)
    elif message == '2':
        # Turn right at place
        setWheelSpeed(leftwheel, forward, 60)
        setWheelSpeed(rightwheel, backward, 60)
    elif message == '3':
        # Forward
        setWheelSpeed(leftwheel, forward, 100)
        setWheelSpeed(rightwheel, forward, 100)
        sleep(200)

```

結論

- ▶ リモコン側の Micro:bit の A ボタンと B ボタンを同時に押すと、車体は直進します。
- ▶ リモコン側の Micro:bit の A ボタンを押すと、車体は左折します。
- ▶ リモコン側の Micro:bit の B ボタンを押すと、車体は右折します。

考察

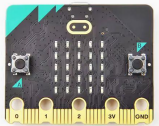
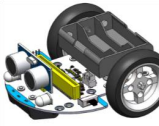
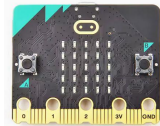
よくある問題

6.8 Micro:bit 加速度センサー無線制御

目的

- ▶ もう 1 台の Micro:bit の加速度センサーを使用して Pybit をワイヤレス操作し、車体の 360 度の方向と速度を制御できます。
- ▶ 両方のメインボードにプログラムする必要があります。

ツール

PC	Micro:bit メインボード	Micro USB ケーブル	Pybit	Micro:bit メインボード
				

プログラミング

リモコンのプログラミング:

```
# Imports go at the top
from microbit import *
import radio

# Display image
display.show(Image.HAPPY)
sleep(400)

# Configure the radio
radio.config(group=1, power=3)
radio.on()

# Code in a 'while True:' loop repeats forever
while True:
    x_strength = accelerometer.get_x()/10
    radio.send('x' + str(x_strength))
    sleep(100)
    y_strength = accelerometer.get_y()/10
    radio.send('y' + str(y_strength))
    sleep(100)
```


Pybit のプログラミング:

```

# Imports go at the top
from microbit import *
import radio

# Car I2C address
i2cAddr = 0x2a

# For wheels
leftwheel = 0
rightwheel = 1
backward = 0
forward = 1
i2cBuf = bytearray([0x00, 0x00])

# Set the wheel speed function
# wheel: 0 = left wheel, 1 = right wheel
# direction: 0 = backward, 1 = forward
# speed: 0--100
def setWheelSpeed(wheel, direction, speed):
    # Important! The speed is between 0 and 100.
    if speed > 100:
        speed = 100
    elif speed < 0:
        speed = 0

    if wheel == leftwheel:
        i2cBuf[0] = 0x05 # left wheel register
        if direction == forward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        elif direction == backward:
            i2cBuf[1] = speed
        i2c.write(i2cAddr, i2cBuf)

    if wheel == rightwheel:
        i2cBuf[0] = 0x06 # right wheel register
        if direction == forward:
            i2cBuf[1] = speed
        elif direction == backward:
            # speed value, 101 is the default required data.
            i2cBuf[1] = speed + 101
        i2c.write(i2cAddr, i2cBuf)

```

```

# Display image
display.show(Image.HAPPY)
sleep(400)

# Configure the radio
radio.config(group=1)
radio.on()

xData = 0
yData = 0

# Code in a 'while True:' loop repeats forever
while True:
    message = radio.receive()
    if message != None:
        # Gets the first character of a string.
        xy = message[0]
        # Gets the second to last character of a string.
        xyData = message[1:]
        # Converts a string to a number.
        xyData = int(xyData)

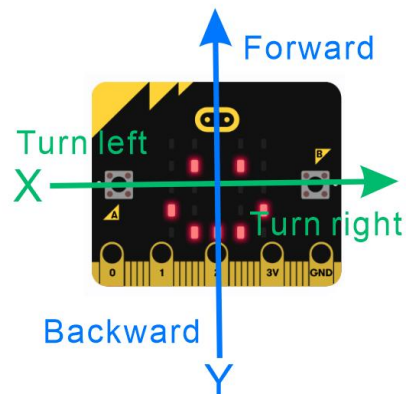
        if xy == 'x':
            xData = xyData
        if xy == 'y':
            yData = xyData

    if yData + xData >= 0:
        setWheelSpeed(leftwheel, forward, yData + xData)
    if yData + xData < 0:
        setWheelSpeed(leftwheel, backward, abs(yData + xData))
    if yData - xData >= 0:
        setWheelSpeed(rightwheel, forward, yData - xData)
    if yData - xData < 0:
        setWheelSpeed(rightwheel, backward, abs(yData - xData))

```

結論

- ▶ リモコン側の micro:bit を傾ける方向に応じて、Pybit の前進方向を制御します。
- ▶ リモコン側の micro:bit の傾き角度に応じて、Pybit の速度を制御します。

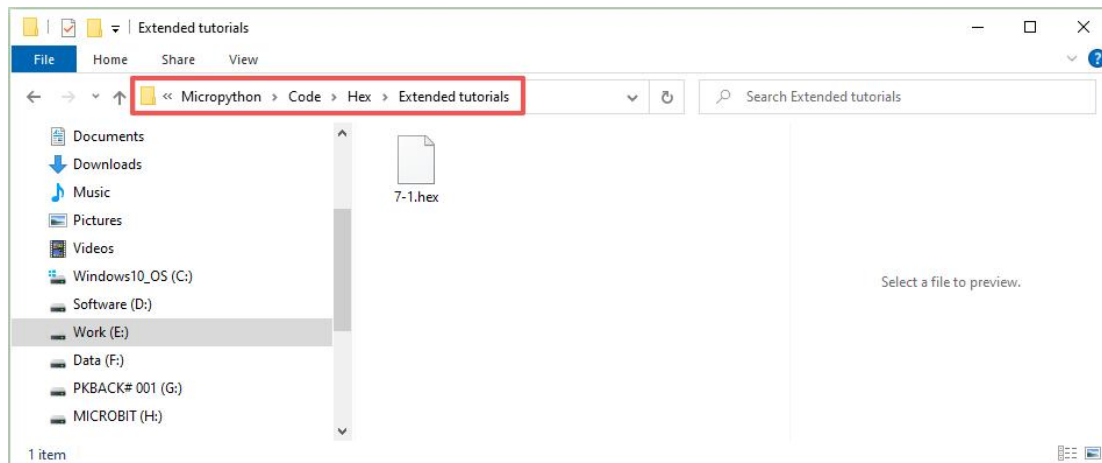


考察

よくある問題

7. Pybit 拡張チュートリアル

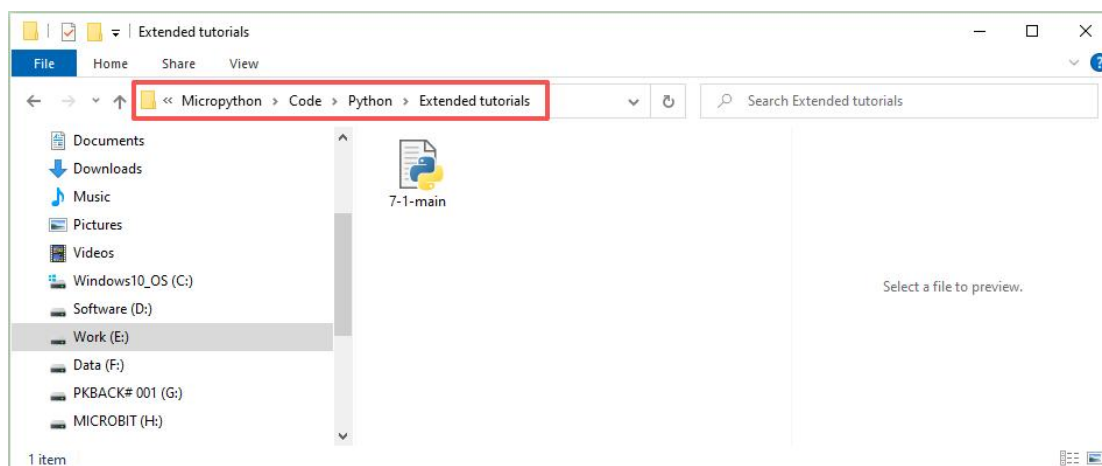
拡張チュートリアルのサンプルプロジェクトは全て、「Micropython -> Code -> Hex -> Extended tutorials」フォルダに保存されています。



特記事項!!!

当社が提供するサンプルプロジェクトをインポートする場合は、
第 4.3 章を参照してください。新規にプロジェクトを作成する場
合は、第 4.1 章を参照してください。以下の授業で使用する **LEGO**
部品はご自身でご購入いただく必要があり、本キットには含まれ
ておりません!!!

拡張チュートリアルのサンプルプロジェクトはすべて、「Micropython -> Code -> Python -> Extended tutorials」フォルダに保存されています。

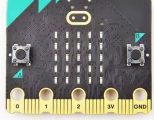
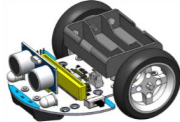


7.1 サーボ駆動

目的

LEGO サーボ、または一般的なサーボモーターを駆動する。

ツール

PC	Micro:bit v2.x.x	Micro USB ケーブル	Pybit	LEGO サーボ
				

プログラミング

```
# Imports go at the top
from microbit import *

# Car I2C address
i2cAddr = 0x2a
i2cBuf = bytearray([0x00, 0x00])

# Servo type
servo90 = 0
servo180 = 1
servo270 = 2
servo360 = 3
pinS1 = 0
pinS2 = 1
pinS3 = 2

# Set the Angle function of 90, 180 and 270 servo.
# index: 0 = S1 pin, 1 = S2 pin, 2 = S3 pin
# servoType: 0 = 90 servo, 1 = 180 servo, 2 = 270 servo
# angle: 90 servo -> 0-90, 180 servo -> 0-180, 270 servo -> 0-270
def setServo(index, servoType, angle):
    angleMap = 0
    if servoType == servo90:
        # Map 0-90 to 50-200
        angleMap = scale(angle, from_=(0, 90), to=(50, 200))
    if servoType == servo180:
        # Map 0-90 to 50-200
        angleMap = scale(angle, from_=(0, 180), to=(50, 200))
    if servoType == servo270:
```

```

    # Map 0-90 to 50-200
    angleMap = scale(angle, from_=(0, 270), to=(50, 200))

    if index == pinS1:
        i2cBuf[0] = 0x0d # S1 pin
    if index == pinS2:
        i2cBuf[0] = 0x0e # S2 pin
    if index == pinS3:
        i2cBuf[0] = 0x0f # S3 pin

    i2cBuf[1] = angleMap
    i2c.write(i2cAddr, i2cBuf)

# Set the speed of 360 servo
# index: 0 = S1 pin, 1 = S2 pin, 2 = S3 pin
# speed: -100 to +100
def setServo360(index, speed):
    # Map -100 - 100 to 0 - 180
    angle = scale(speed, from_=(-100, 100), to=(0, 180))
    setServo(index, servo180, angle)

# Code in a 'while True:' loop repeats forever
while True:
    # The 180 degree servo of the S1 pin turns to the 0 degree position
    setServo(pinS1, servo180, 0)
    # The 360-degree servo of the S3 pin turns forward at the maximum speed
    setServo360(pinS3, 100)
    sleep(1000)
    # The 180 degree servo of the S1 pin turns to the 180 degree position
    setServo(pinS1, servo180, 180)
    # The 360-degree servo of the S3 pin turns backward at
    # the maximum speed
    setServo360(pinS3, -100)
    sleep(1000)

```

結論

サーボモーターが 0～180 度の間で往復動作します。

考察

90 度と 270 度のサーボモーターを駆動する方法

よくある問題

8. Q&A

8.1 Micro:bit にコードをアップロードできません

👉 データ通信対応の USB ケーブルをご使用ですか？

👉 USB ポート（接続）は正常ですか

8.2 Micro:bit のドライブレター表示が異常です

👉 これは、誤って micro:bit のリセットボタンを押した状態で電源を投入したため、micro:bit がファームウェア更新モードに移行したことが原因です。ファームウェアを再更新すると正常に復旧します。詳細は以下のリンクを参照してください：

<https://microbit.org/get-started/user-guide/firmware/>

8.3 コードを再アップロード後もモーターが回転し続ける

👉 原因：第 5.6 章にある「モーター停止関数」を再呼び出ししていないためです。

```
setWheelSpeed(wheel, direction, 0)
```

8.4 モーターとヘッドライトの両方が作動しない

- 🔧 micro:bit が Pybit のスロットに完全に挿入されていることを確認してください。
- 🔧 micro:bit のエッジコネクタが清潔であることを確認してください。
- 🔧 Pybit のスロットが清潔であることを確認してください。



- 🔧 第6.1章のよくある問題（FAQ）を参考に、車輪の補正値を0に再設定してください。

8.5 その他の故障

- 🔧 組み立てが正しいかご確認ください。
- 🔧 バッテリー残量が十分かご確認ください。
- 🔧 使用中のバッテリーが仕様に適合しているかご確認ください。

9. お問い合わせ

私共はオープンソースハードウェアを用いたメイカー教育に携わり、心を込めて開発した製品を皆様にお届けしております。そして、皆様からご評価をいただければと願っております。

製品の改善点について率直なご意見をいただき、積極的に対話を重ねながら、ぜひ私たちのチームの一員としてご参加いただけますよう、心よりお待ちしております。

✉ support@siyeenove.com

 <https://siyeenove.com>